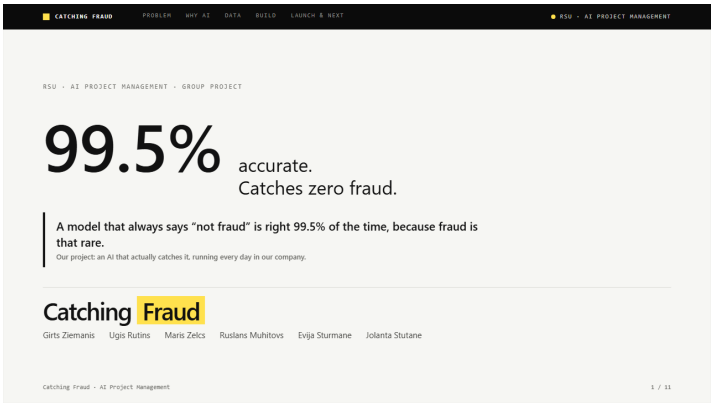
SLIDE 1 / 11 · RSU · AI PROJECT MANAGEMENT · GROUP PROJECT

Hook



WHAT TO SAY

- (~20 s) "Imagine a fraud detector that is 99.5% accurate. Sounds great? It catches zero fraud."
- Only about 0.5% of card payments are fraud, so a model that always says "not fraud" is right 99.5% of the time, and useless.
- "We are the AI team of a card issuer. In the next minutes we show you how our AI catches fraud every day: our data, why it needs AI, how we built it, and how it runs."

BACKGROUND

The setting. We present ourselves as the AI team of a card-issuing company (an imagined company, as the brief allows). The process we show (daily data, real-time scoring, analyst reviews, nightly retraining by an AI agent) is how a modern company would run fraud detection. The model results we quote were measured on our prototype dataset, which is said once on the data slide.

Why accuracy misleads here. In our prototype data only 1,782 of 339,607 payments are fraud (0.52%). A model that answers "not fraud" every time is correct 99.48% of the time, yet never stops a single fraudster. Accuracy counts every correct answer equally, so when one class is tiny the big class dominates the score.

What we measure instead. Recall (the share of real fraud we catch), false alarms as a guardrail (we must not block every customer), and money as the business measure. Every number later in the pitch is one of these three.

About the project (read this first). This is our group project for RSU's AI Project Management course: one AI project shown across its whole life, from the problem to running it in a company, in a 5–7 minute pitch. Our project is an AI that detects card fraud for a card-issuing company. Two pieces of real work sit behind the slides: a machine-learning notebook (we explored a public card-payment dataset and tested five models on it) and Ģirts' pipeline application, which runs the chosen model (LightGBM) behind an API and shows every step a payment goes through. Every number on the slides comes from one of these two.

The storyline. Problem (slide 2) → why AI and not simple rules (3) → the data and what fraud looks like in it (4–5) → how we built and chose the model (6) → the pipeline, following one real test payment (7–10) → launch, measuring success, risks and next steps (11). At the timings given in each slide's bullets the pitch takes about 6 minutes.

How to run the deck. Open index.html in a browser (Chrome or Edge). The arrow keys, Space and PageUp/PageDown move between slides; F switches to full screen; N shows these notes on screen. On slides 7–9 you can click a step box to open a pop-up with that step's data; close it with "× close" or by clicking outside it.

On the screen. The headline "99.5% accurate. Catches zero fraud.", one line explaining it, the project name and the team. Read the headline, pause for a second, then give the explanation: it is the hook for the whole pitch.

If asked: is the company real? No. The brief allows a real or imagined project, so we present ourselves as the AI team of an imagined card issuer and describe how a modern company would run this. The model, the pipeline and all results are real: they were built and measured on our prototype dataset.

On the screen. The six boxes, left to right, are the first six steps every payment goes through, grouped as Intake, Preprocessing and Model. The purple line in each box is the real value for our test payment. The card at the bottom left is the test payment itself: an online purchase at Best Buy for $2,334.03 on a Saturday at 23:29 by a 78-year-old cardholder from Seattle.

What each pop-up shows (when you click a box). 01: the raw request fields (merchant, category, amount, time, cardholder). 02: all 10 fields the model needs are present, so the request is accepted. 03: the code that creates hour, day of week, age, distance and log-amount, with the results (hour 23, age 78, distance 1,757 km, log-amount 7.7558) and the raw fields that are dropped. 04: the 27 yes/no columns as a grid with two switched on (online shopping, state WA), giving 39 inputs in total. 05: why the scaling step is folded into the model. 06: the path this payment takes through the first of the 300 trees, how the score grows as trees are added, and the final raw score, +6.4959.

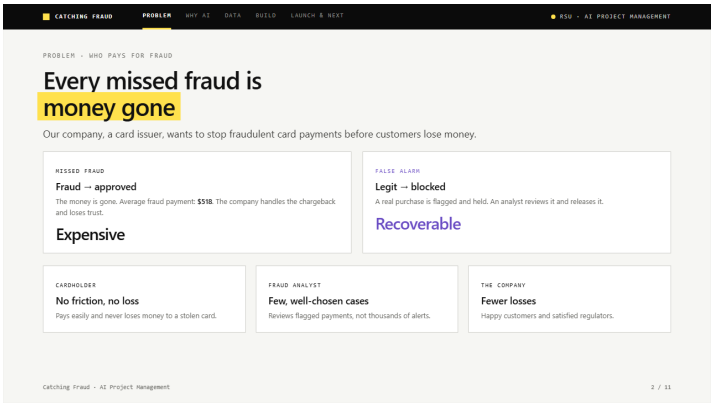
Suggested click. If time allows, open only 03 (feature engineering): it shows the patterns from slide 5 turning into numbers. Close the pop-up before moving on.

Words to know. API: the service that other systems call with a payment and that answers with a verdict. Request: the message the payment system sends to the API. One-hot encoding: turning a category such as "shopping_net" into yes/no columns the model can read. Log-odds (raw score): the model's internal score; 0 means 50/50, and the higher it is, the more likely fraud.

If asked: why a US city and dollars? The prototype dataset is a public US-style dataset, so amounts are in $. Its coordinates are synthetic and do not match the cities, so we show only the city, the state and the distance.

SLIDE 2 / 11 · PROBLEM · WHO PAYS FOR FRAUD

The problem



WHAT TO SAY

- (~40 s) Two possible mistakes with very different prices. A missed fraud is real money lost (on average $518) plus a chargeback and lost trust. A false alarm is a genuine payment held until an analyst releases it.
- So our goal: catch as much fraud as possible, while keeping false alarms low. "When in doubt, stop the payment."
- Three users: the cardholder, the fraud analyst who reviews cases, and the company, which pays for the losses.

BACKGROUND

Two kinds of mistakes. A *missed fraud* (false negative) means a fraudulent payment is approved. A *false alarm* (false positive) means a genuine payment is blocked. A fraud model always trades one against the other: catching more fraud usually means more false alarms.

Why the prices differ. In our prototype data the average fraudulent payment is $518 (median $356), against $68 for a normal payment. On top of the lost money there is the chargeback process and the risk of losing the customer. A false alarm costs a moment of friction, which is recoverable, but too many of them drive customers away, so false alarms are a guardrail.

Course link. Lecture 6: "a wrong blur is cheap; a missed face is not." This cost difference drives every later decision: the metric, the model choice, the threshold and the manual review step.

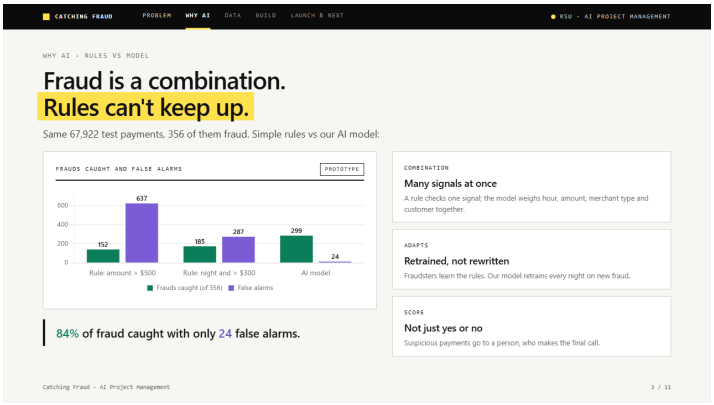
On the screen. Left: the two mistakes a fraud model can make. A missed fraud (fraud approved) is expensive; a false alarm (a genuine payment blocked) is recoverable. Right: the three users and what each of them wants: the cardholder, the fraud analyst and the company.

Words to know. Card issuer: the bank or company that gives the customer the card and carries the loss when a fraudulent payment is approved. Chargeback: the formal process of reversing a disputed card payment; it costs the issuer time and money. False negative and false positive: the technical names for a missed fraud and a false alarm.

If asked: why not simply block everything suspicious? Every blocked genuine payment annoys a customer and costs an analyst's time, and the analyst team is small. The model has to send them a few well-chosen cases, not thousands of alerts. That is why we watch false alarms as closely as caught fraud.

SLIDE 3 / 11 · WHY AI · RULES VS MODEL

Why AI



WHAT TO SAY

- (~45 s) Why do we need AI at all? We tested the obvious rules on exactly the same test payments as the model.
- "Block every payment over $500" catches 152 of 356 frauds (43%) and blocks 637 innocent customers. "Block night payments over $300" catches 185 (52%) with 287 false alarms.
- Our AI model catches 299 (84%) with only 24 false alarms. It combines many weak signals that a rule looks at one by one.
- And fraud changes: rules are rewritten by hand, our model is retrained every night. It also gives a score, so suspicious payments go to an analyst.

BACKGROUND

How we tested the rules. Both rules were applied to the same locked test set of our prototype dataset as the model: 67,922 payments, 356 of them fraud. The night window is 22:00–03:59.

Why the model wins. A rule looks at one or two signals and draws a hard line. Fraud rarely shows up in one signal; it shows up in combinations, for example a mid-sized online purchase late at night by an older customer. The model weighs all inputs together and learns these combinations from past payments.

Why it matters over time. Fraudsters adapt to whatever gets blocked. A rule set has to be rewritten by hand; a model is retrained on newly confirmed fraud. The score from 0 to 1 also lets the company treat payments differently: low-risk payments pass smoothly, flagged ones go to an analyst for review.

Honest caveat. These are simple illustrative rules, not a tuned rule engine. Real rule sets are stronger, but they hit the same limit: every rule is written by a person and checks a fixed combination.

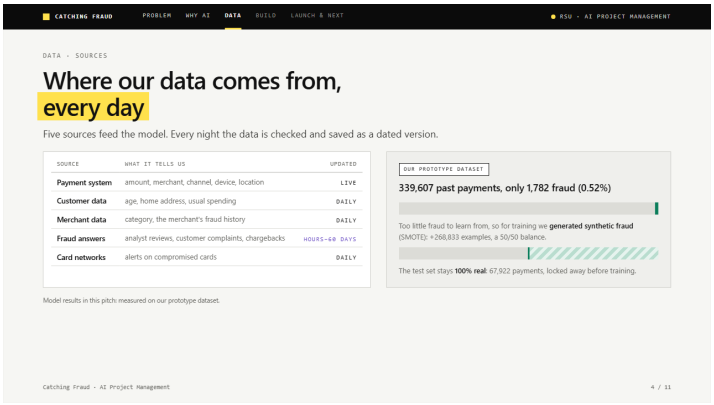
On the screen. The chart compares three approaches on the same 67,922 test payments (356 of them fraud). Green bars are frauds caught, purple bars are false alarms. Rule 1, amount over $500: 152 caught, 637 false alarms. Rule 2, night and over $300: 185 caught, 287 false alarms. Our AI model: 299 caught, 24 false alarms. Point at the model's very short purple bar: that is the key message. The three cards on the right give the three reasons: combination, adapts, score.

Words to know. Test set: past payments the model never saw while it was trained, kept aside so it can be judged fairly. Score: the model's fraud probability for a payment, between 0 and 1.

If asked: could better rules do the same? Better rules would get closer, but someone has to write and maintain every rule, and fraudsters learn to avoid them. The model learns the combinations from data and is retrained automatically. Rules still have a place as a fallback if the model is unavailable.

SLIDE 4 / 11 · DATA · SOURCES

Where the data comes from



WHAT TO SAY

- (~50 s) Our model is fed by five sources. The payment system streams every payment live; customer, merchant and card-network data refresh daily. Our prototype shows the payment stream and the feedback labels; the other sources are the production plan.
- The fraud answers are the labels the model learns from. They arrive with delays: analyst reviews the same day, customer complaints within days, chargebacks after up to 60 days.
- Every night the data is checked and saved as a dated version, so we always know which data trained which model.
- We built the model on a prototype dataset of 339,607 past payments. Only 1,782 are fraud, so we generated synthetic fraud for training. The test set stays 100% real.

BACKGROUND

Live vs daily. Payment data must be live because the model scores each payment while the customer waits. Customer, merchant and network data change slowly, so a nightly refresh is enough. Lecture 2: information saved at capture costs nothing; added a month later it costs days, so every payment is stored with its score, model version and decision.

Labels with delays. Fraud is only confirmed later. Analyst reviews of flagged payments give same-day labels, customer complaints about approved payments reveal missed fraud within days, and chargebacks arrive after 30–60 days. The model can only be judged and retrained on payments whose answers have arrived (Lecture 8: who holds the answer sheet, and when).

The nightly job. Load the day's payments, run data checks (missing fields, changed formats, unusual volumes; a failed check stops the run and alerts the data engineer), refresh customer profiles, attach new fraud answers, save a dated dataset version (Lecture 2: which exact data trained the model?).

The prototype dataset. A public dataset with the same structure as card-payment data: 339,607 payments, 2019–2020, 15 columns, no missing values. 20% was locked away as the test set (67,922 payments, 356 frauds), stratified and with a fixed seed. The training set (271,685 payments, 1,426 frauds) was balanced with SMOTE: each synthetic fraud is placed between a real fraud and one of its five most similar frauds, giving 270,259 of each class. XGBoost and LightGBM used class weights instead. If asked: the prototype is a public dataset, because a student team cannot use real customers' data.

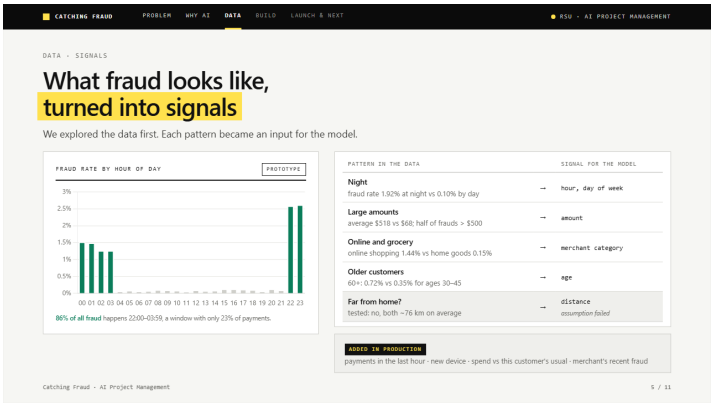
On the screen. The table lists the five data sources a real card issuer would feed the model, what each tells us and how often it updates. The box below is the prototype dataset we actually trained and tested on, and how we balanced it.

Words to know. Label: the right answer (fraud or not) for a past payment; the model learns from labelled examples. SMOTE (Synthetic Minority Over-sampling Technique): creates artificial fraud examples between similar real ones, so the model sees enough fraud while it learns. Data version: a dated snapshot of the data, so every model can be traced back to the exact data that trained it.

If asked: does synthetic data make the results look better? No. Synthetic examples are used only for training. Every result in the pitch is measured on the test set, which is 100% real payments and was locked away before any training started.

SLIDE 5 / 11 · DATA · SIGNALS

What fraud looks like



WHAT TO SAY

- (~45 s) Before building any model, we looked at what makes fraud different, and turned each finding into a signal.
- Night: 86% of fraud happens between 22:00 and 04:00. Amount: fraud averages $518 vs $68. Online shopping and grocery have the highest fraud rates. Customers over 60 are targeted twice as often as 30–45 year olds.
- We also tested "fraud happens far from home": the data said no. Lesson: test assumptions with data.
- In production we add behaviour signals our live data gives us: how many payments in the last hour, a new device, spending far above the customer's usual.

BACKGROUND

From pattern to signal. This step is called feature engineering: the 15 raw columns of the prototype data became 39 model inputs. All numbers on this slide are computed from the full prototype dataset.

The findings. Night: fraud rate 1.92% at night vs 0.10% in the daytime, about 19 times higher. Amount: medians $356 vs $46; 47% of frauds are above $500; the model also uses the logarithm of the amount to tame the range up to $28,000. Category: online shopping 1.44%, in-store grocery 1.32%, online miscellaneous 1.28% vs 0.15% for home goods. Age: 0.72% for customers over 60 vs 0.35% for 30–45 year olds.

The failed assumption. We calculated the distance between home and merchant for every payment: 75.6 km on average for both fraud and normal payments. We kept the feature, but the lesson is to test intuition with data.

Added in production. Live data gives behaviour signals the prototype data does not have: velocity (payments per card in the last hour), a new device or new country, spending compared with this customer's usual pattern, and the merchant's recent fraud rate. These are the strongest signals in modern fraud detection, because most fraud today is online and mobile, where a stolen card behaves differently from its owner. They are part of the company plan, not of our prototype results.

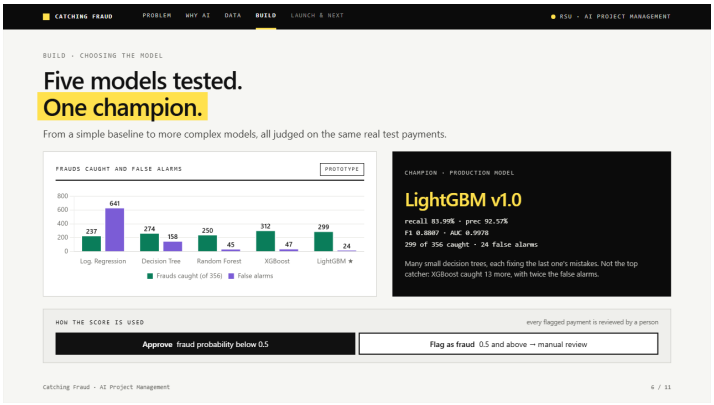
On the screen. The chart shows the fraud rate for every hour of the day; the tall bars between 22:00 and 03:00 are the night window. The table turns each pattern (left) into a model input (right). The strip at the bottom lists the signals we would add with live company data.

Words to know. Feature (also called input or signal): a number the model reads for every payment, for example the hour or the amount. Log of the amount: a way to compress very large amounts, so that one $28,000 payment does not dwarf everything else.

Tip for the presenter. The distance row is the honest moment of the pitch: we expected fraud to happen far from home, and the data said no. Say it with confidence; it shows that we test our assumptions. The same finding comes back on slide 8, where distance pushes our test payment towards legitimate.

SLIDE 6 / 11 · BUILD · CHOOSING THE MODEL

How we built it



WHAT TO SAY

- (~50 s) Lecture 3: start simple, change one idea at a time. We trained five models, from a simple straight-line model to tree ensembles, and judged all on the same locked test set.
- The baseline caught 237 frauds with 641 false alarms. The champion, LightGBM, caught 299 (84%) with only 24 false alarms.
- We chose on balance: XGBoost caught 13 more, but with twice the false alarms.
- The model gives a fraud probability: below 0.5 the payment is approved, at 0.5 and above it is flagged as fraud and reviewed by an analyst.

BACKGROUND

The approach (Lecture 3). Logistic Regression tests a single straight-line rule; Decision Tree tests yes/no rule combinations; Random Forest a vote of many trees; XGBoost trees that correct each other's mistakes; LightGBM the same idea, built for speed. Each model was tuned automatically with 5-fold cross-validation on the training data only, then judged once on the locked test set of our prototype dataset.

Results (Recall / precision / F1): Logistic Regression 0.67 / 0.27 / 0.38; Decision Tree 0.77 / 0.63 / 0.70; Random Forest 0.70 / 0.85 / 0.77; XGBoost 0.88 / 0.87 / 0.87; LightGBM 0.84 / 0.93 / 0.88, ROC-AUC 0.998. Precision is the share of alarms that were real fraud; F1 balances Recall and precision.

Why LightGBM. Best balance (highest F1) and fewest false alarms. With only 356 fraud cases, Recall is uncertain by about ±4 points (≈ ±14 frauds), so XGBoost's 13 extra catches could be luck; the choice is a business trade-off to confirm in shadow mode.

From score to decision. The production pipeline flags a payment at a fraud probability of 0.5 or more and sends it to manual review. In the notebook, lowering the cutoff to 0.4 caught 304 instead of 299 frauds for 28 instead of 24 false alarms (highest F1, 0.884); this is a candidate change for a future release, to be confirmed on live data. Under EU payment rules (PSD2), low-risk payments may also skip extra authentication, so a good score means fewer checks for honest customers.

On the screen. The chart shows the five models from simplest to most complex, with frauds caught (green) and false alarms (purple): Logistic Regression 237 caught / 641 false alarms, Decision Tree 274 / 158, Random Forest 250 / 45, XGBoost 312 / 47, LightGBM 299 / 24. The dark card on the right is the champion, LightGBM v1.0, the model running in the pipeline. The band at the bottom shows how its score is used: approve below 0.5, flag as fraud at 0.5 and above, and a person reviews every flagged payment.

Words to know. Recall 83.99%: 299 of the 356 frauds were caught. Precision 92.57%: 299 of the 323 alarms were real fraud, so 24 were false alarms. F1 (0.8807): one number that balances recall and precision. AUC (0.9978): how well the model ranks fraud above normal payments, where 1.0 is perfect. Cross-validation: tuning each model on several splits of the training data, so that the test set is used only once, at the end.

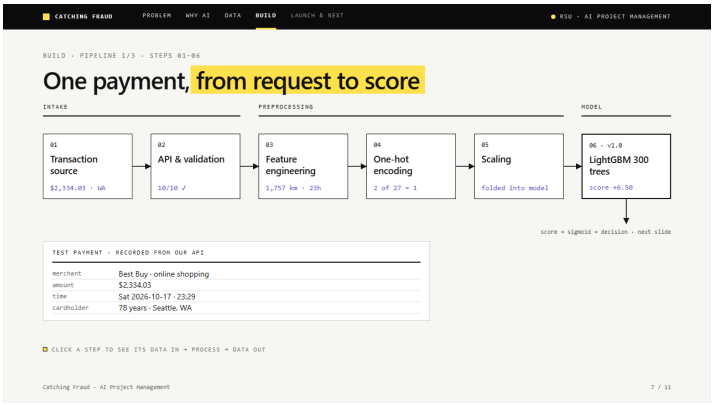
If asked: what is LightGBM? A model made of many small decision trees (300 in ours), built one after another; each new tree corrects the mistakes of the trees before it. This is called gradient boosting. It is fast and works

very well on table data such as payments.

If asked: why not XGBoost, it caught more? XGBoost caught 13 more frauds but made about twice as many false alarms (47 vs 24). With only 356 frauds in the test set, a difference of 13 can be chance, so we chose the better balance and would confirm the choice on live payments in shadow mode.

SLIDE 7 / 11 · BUILD · PIPELINE 1/3 · STEPS 01–06

AI pipeline



WHAT TO SAY

- (~40 s) "Let's follow one real test payment through the system: an online purchase at Best Buy, $2,334, at 23:29, by a 78-year-old cardholder."
- The API first checks the request, then turns raw fields into the signals the model learned: hour 23, age 78, distance, log-amount. Categories become 27 yes/no columns, 39 numbers in total.
- LightGBM's 300 small trees add up a score: +6.50. Click any box to show the real data at that step.

BACKGROUND

What is shown. One recorded response of our fraud API (POST /explain) for a single synthetic test payment; the slides do not call the API. Source: Ģirts' pipeline_example.json.

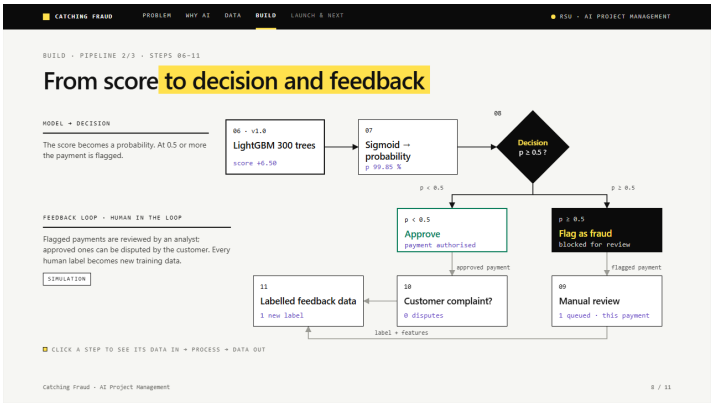
How the score is built. Each of the 300 trees adds a small correction to a starting value (−9.23 in log-odds); the total for this payment is +6.50.

Scaling (05). The model was trained on standardised inputs, but trees only compare a value with a threshold, so the scaler was folded into the thresholds: identical predictions, and the API feeds raw values.

Lecture 6 mapping. These slides are the "serve" step (train → export → catalogue → build → deploy → serve → watch); "watch" and retraining follow on the next two slides.

SLIDE 8 / 11 · BUILD · PIPELINE 2/3 · STEPS 06–11

AI pipeline: decision and feedback



WHAT TO SAY

- (~40 s) The score goes through a sigmoid and becomes a probability: 99.85% fraud. The rule is simple: at 0.5 or more the payment is flagged and blocked; below 0.5 it is approved.
- Then people close the loop. An analyst reviews every flagged payment; our Best Buy payment is confirmed as fraud. Approved payments can still be disputed by the customer, which reveals a fraud the model missed.
- Every human answer becomes a labelled training example. In our prototype this loop is simulated.

BACKGROUND

Sigmoid (07). p = 1/(1+e^−score); for a score of +6.4959 this gives 0.998493, far above the 0.5 cut-off.

Why it was flagged. Open step 08: mostly the amount (+10.44), the late hour (+3.18) and the age (+1.97). Distance actually pushed towards legitimate (−0.74), the same lesson as on the data slide. These are SHAP contributions computed by LightGBM itself (pred_contrib), and they add up exactly to the score.

Human in the loop (Lecture 6). People's daily work teaches the next model: analyst decisions on flagged payments and customer disputes on approved ones are both stored as labelled feedback data.

On the screen. The top row continues from the previous slide: the score (06) becomes a probability (07), and the decision diamond (08) splits the flow. Left branch, p < 0.5: Approve. Right branch, p ≥ 0.5: Flag as fraud. The bottom row is what people do afterwards: the analyst's manual review (09) and the customer's complaint (10) both produce labels, which are stored as feedback data (11).

Our payment's route. 99.85% → Flag as fraud → manual review (09) → the analyst confirms fraud → 1 new label in 11. The Approve → complaint route is the other path, for frauds the model misses; our payment does not take it.

What each pop-up shows. 06: the score again. 07: the sigmoid formula with our numbers. 08: the 0.5 rule plus the five strongest reasons as bars (amount +10.44, hour +3.18, age +1.97, distance −0.74, category +0.37). Approve and Flag: what happens on each branch. 09: what the analyst checks and the label recorded. 10: how a customer dispute becomes a label. 11: the training row that is stored.

Suggested click. Open 08: it shows why the model flagged the payment, which is the most convincing moment for the audience. Point out that distance pushed towards legitimate, as on slide 5.

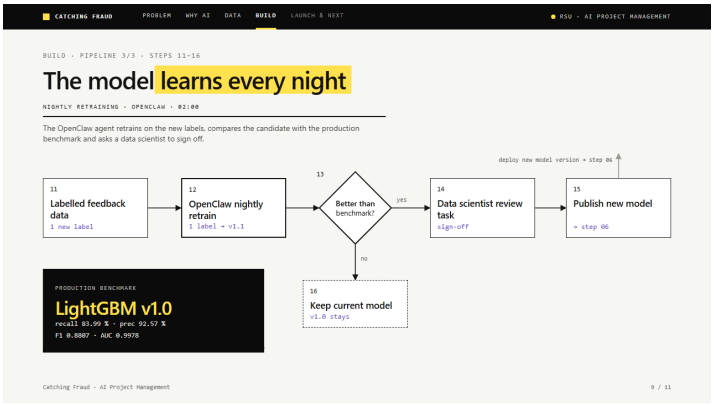
Words to know. Sigmoid: a formula that turns any score into a probability between 0 and 1. SHAP contributions: how much each input pushed the score up (towards fraud) or down (towards legitimate). Human in the loop: people check the AI's decisions, and their answers teach the next version. The "Simulation" tag: in our prototype these human steps are simulated; in the company they would be real analyst and customer actions.

If asked: what if the analyst is wrong? Labels arrive from several sources over time (analyst reviews, customer complaints, chargebacks), so a wrong label can be corrected later. How often analysts overturn the model is also

one of our KPIs on slide 11.

SLIDE 9 / 11 · BUILD · PIPELINE 3/3 · STEPS 11–16

AI pipeline: nightly retraining



WHAT TO SAY

- (~40 s) Every night at 02:00 the OpenClaw AI agent retrains a candidate model on the new labels.
- The candidate is compared with the production benchmark, LightGBM v1.0. Only if it is better, and a data scientist signs off, is it published back to step 06. Otherwise we keep the current model.
- Next: the whole pipeline in one view.

BACKGROUND

Why a gate. An automatic retrain can make the model worse (bad labels, a strange week). The benchmark check (13) and a human sign-off (14) stop a worse model from going live; the previous version is kept for rollback.

Benchmark. The production model LightGBM v1.0: recall 83.99%, precision 92.57%, F1 0.8807, AUC 0.9978.

Lecture 6 mapping. This is the "watch" and retrain part of the MLOps loop: monitor, collect labels, retrain, validate, deploy.

On the screen. Left to right: the new labels (11) go to OpenClaw, an AI agent that retrains a candidate model every night at 02:00 (12). The diamond (13) compares the candidate with the production benchmark. Yes: a data scientist reviews it (14), it is published (15), and the arrow shows it replacing the model at step 06. No: we keep the current model (16). The dark card at the bottom left is the benchmark the candidate has to beat.

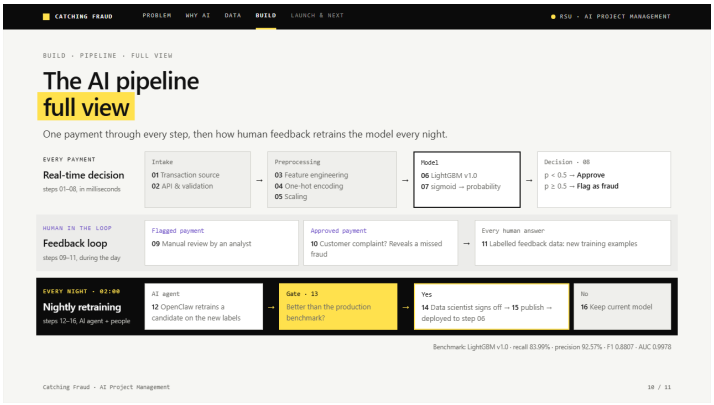
What each pop-up shows. 11: the stored labels. 12: the training settings (300 trees, learning rate 0.05, depth 7, 31 leaves) and the candidate, v1.1. 13: both models scored on the same 67,922 test payments, and the rule for yes and no. 14: what the data scientist checks before signing off. 15: how the model is published together with its 39 input columns. 16: what happens when the candidate loses.

Words to know. AI agent: software that carries out a multi-step task on its own; here it collects the new labels, retrains, tests and creates a review task. Benchmark: the scores of the model now in production, the bar to beat. Candidate and champion: the new model and the one in production. Rollback: returning to the previous model if something goes wrong.

If asked: why not let the agent publish on its own? A worse model either blocks real customers or lets fraud through. One human sign-off per release is cheap insurance and keeps a person accountable, which is a central theme of the course: AI with human oversight.

SLIDE 10 / 11 · BUILD · PIPELINE · FULL VIEW

Pipeline: full view



WHAT TO SAY

- (~30 s) Here is the whole pipeline in one view, steps 01 to 16. It has three loops.
- Every payment: the transaction comes in, is validated, turned into model inputs, scored by LightGBM, and the decision is made. Below a probability of 0.5 it is approved, above it is flagged as fraud.
- During the day, people close the loop: an analyst reviews every flagged payment, and a customer complaint about an approved payment reveals a fraud we missed. Every human answer becomes a new training example.
- Every night at 02:00 an AI agent retrains a candidate model on the new labels. Only if it beats the production benchmark, and a data scientist signs off, is it published. Otherwise we keep the current model. Next: how we launch it and measure success.

BACKGROUND

Source. This slide sums up the three pipeline slides before it on one page; the step numbers 01–16 are the same, so the audience can follow along.

Real-time decision (01–08). Intake: the transaction arrives from the payment system and the API validates it (all required fields present). Preprocessing: feature engineering (for example distance to the merchant and hour of the day) and one-hot encoding of categories; scaling (05) is folded into the model thresholds in the API. Model (06): LightGBM returns a raw score, which the sigmoid (07) turns into a fraud probability. Decision: approve below 0.5, flag as fraud (blocked for review) at 0.5 and above. In the notebook a cutoff of 0.4 caught 5 more frauds for 4 more false alarms; that is a candidate change for a future release.

Feedback loop (09–11), human in the loop. Lecture 6: people's daily work teaches the next model. Flagged payments get a manual analyst review; approved payments can be disputed by the customer, which reveals missed fraud. Both answers are stored as labelled feedback data, with the payment's features.

Nightly retraining (12–16). An AI agent (OpenClaw) retrains a candidate model on the new labels at 02:00 and compares it with the production benchmark, LightGBM v1.0 (Recall 83.99%, precision 92.57%, F1 0.8807, AUC 0.9978). If the candidate is better, a data scientist gets a review task and signs off before the model is published and deployed to step 06. If not, the current model stays. This links to the agent lectures (L10–11) and to Lecture 3's champion loop: a challenger replaces the champion only when it wins.

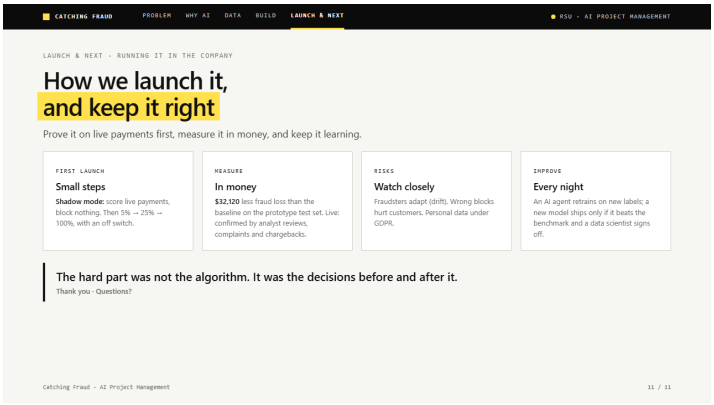
Good question to expect: "how much better is better?" With 356 fraud cases in the test set, Recall is uncertain by about ±4 points, so the gate should require a clear margin, not any improvement, and big changes can still go through shadow mode before full release (Lecture 8).

On the screen. The same 16 steps on one page, in three lanes. Top: every payment, in milliseconds. Middle: people during the day. Bottom: the nightly loop. Use it as a short recap, or come back to it when questions about the pipeline come up.

Timing tip. If you are running late, show this slide for one sentence ("this is the whole loop on one page") and move on; the three slides before it carry the content.

SLIDE 11 / 11 · LAUNCH & NEXT · RUNNING IT IN THE COMPANY

Launch and next steps



WHAT TO SAY

- (~50 s) First launch: shadow mode, where the model scores real payments but blocks nothing; then a canary release, 5%, 25%, 100%, always with a way back. After that, updates come through the nightly loop you saw in the pipeline.
- Measure: on the prototype test set the champion prevents about $32,120 more fraud loss than the baseline. Live, success is confirmed by analyst reviews, customer complaints and chargebacks.
- Risks: fraudsters adapt, wrong blocks hurt real customers, and we handle personal data under GDPR.
- Improve: every night an AI agent retrains on the new labels, and a new model only ships if it beats the benchmark and a data scientist signs off. Close with the one-liner, then questions.

BACKGROUND

Launch in small steps (Lecture 8). A test score is only a promise: in the SickKids silent trial a model scored 0.90 on old data but 0.50 on new patients. Shadow mode checks the promise on the company's live payments without affecting customers. Analyst reviews and customer complaints give a first read within days; chargebacks complete the picture after about 60 days (Stripe faces the same delay). The canary release compares the model with today's rules in the same weeks, not before vs after.

Measure in money. On the prototype test set the champion catches 62 more frauds than the baseline; at the average fraud value of $518 that is about $32,120 less loss on 67,922 payments. Live KPIs: fraud caught, false alarms and complaints, losses prevented per month, analyst workload, results per customer group, and how often analysts overturn the model's flags (step 09 in the pipeline gives us this for every flagged payment).

Main risks. Drift: fraud patterns change, so the model ages (Lecture 8: Instacart in 2020). Wrong blocks hurt real customers. Unequal errors: age is one of the strongest signals, so error rates are checked per age group. Privacy: birth dates and home locations fall under GDPR (access by role, deletion on request, a documented model). One bad input, such as a wrong address, can corrupt several signals, so inputs are checked and rules are the fallback.

Improve every night (Lecture 2, the data engine). Every analyst review and customer complaint becomes a label. At 02:00 an AI agent retrains a candidate and compares it with the production benchmark; a data scientist signs off before release, and larger changes can still go through shadow mode first. First targets: study the 57 frauds the prototype missed, and add the behaviour signals from live data.

Closing line. "The hard part was not the algorithm. It was the decisions before and after it." Then open for questions; the backup deck has every detail.

On the screen. Four cards: First launch, Measure, Risks and Improve, then the closing line. Go through them left to right in about ten seconds each.

Words to know. Shadow mode: the model runs on real payments and its decisions are recorded but not used, so we can compare them with what really happened. Canary release: switching the model on for a small share of

payments first (5%, then 25%, then 100%). Drift: the world changes, so a model trained on old data slowly gets worse. GDPR: the EU data protection law. KPI: a key number we track to judge success.

Where the $32,120 comes from. On the test set the champion caught 299 frauds and the simplest model (Logistic Regression) 237: 62 more. 62 × $518.07, the average fraud amount, is about $32,120 less fraud loss on 67,922 payments.

Ending. After the closing line, say "Thank you" and invite questions. The "If asked" paragraphs on the earlier pages cover the most likely questions.